

PKCERT AI & Software Development Internship

Task 19 – Training Loops: Loss Functions, Optimizers (SGD & Adam), and Epoch/Batch Management

Objective: Having built a working feed-forward network using PyTorch's `nn.Module` and `autograd` in Task 18, interns will now move beyond calling `torch.optim` as a black box and develop a rigorous, mechanistic understanding of the training loop itself. This task requires implementing SGD and Adam from scratch to expose exactly what `torch.optim` automates, examining loss functions beyond their default call signatures, and running controlled experiments on batching, scheduling, and gradient handling. Every design choice (loss function, optimizer, batch size, schedule) must be justified with either a mathematical argument or empirical evidence from your own runs — not general convention alone.

Total Marks: 100

Difficulty Level: Hard

Part A – Loss Functions in Depth (20 Marks)

- Implement at least three loss functions from scratch using only raw tensor operations (no `torch.nn.functional`) — e.g., MSE, Cross-Entropy, and L1/Hinge loss — and verify each against its PyTorch built-in equivalent to within a stated numerical tolerance.
- Explain mathematically why cross-entropy is preferred over MSE for classification, referencing gradient magnitude and vanishing-gradient behavior when the softmax output saturates near 0 or 1.
- Implement a custom loss combining a data term with an L2 (weight-decay-style) regularization penalty; derive its gradient by hand and confirm that `autograd`'s computed gradient matches your derivation.
- Discuss the effect of class imbalance on standard cross-entropy loss, and implement a weighted variant (e.g., class-weighted cross-entropy or focal loss) that corrects for it. Demonstrate the difference in per-class gradient contribution before and after weighting.

Part B – Optimizers: SGD and Adam Internals (30 Marks)

- Implement vanilla SGD, and SGD with momentum, entirely from scratch as a manual parameter-update loop (no `torch.optim`). Verify that your implementation's parameter trajectory matches `torch.optim.SGD` step-for-step on a small convex toy function (e.g., a quadratic bowl).
- Implement Adam from scratch, including the first and second moment estimates and their bias correction terms. Verify it against `torch.optim.Adam` on the same toy function, and explain any small trajectory divergence in terms of epsilon placement or default hyperparameter differences.
- Derive, with a worked numerical example, why Adam's per-parameter adaptive learning rate can converge faster than vanilla SGD on sparse or unevenly-scaled gradients.
- Run a controlled comparison of SGD, SGD+momentum, and Adam on the same network, same initialization (fixed random seed), and same data. Plot all three loss curves on one graph and discuss convergence speed, stability of the descent, and final validation accuracy.
- Explain the difference between L2 regularization inside Adam and true weight decay as implemented in AdamW. Demonstrate numerically, on a toy example, that the two are not equivalent for an adaptive optimizer, even though they are equivalent for plain SGD.

Part C – Epochs, Batches, and Training Loop Engineering (35 Marks)

- Explain the distinction between batch gradient descent, mini-batch gradient descent, and stochastic gradient descent, and implement all three on the same dataset and model so their update dynamics can be directly compared.
- Use `torch.utils.data.Dataset` and `DataLoader` to implement proper batching and per-epoch shuffling. Explain why re-shuffling the data each epoch affects generalization and helps avoid spurious ordering effects.
- Design and run an experiment varying batch size (e.g., 8, 32, 128, and full-batch) while holding all other hyperparameters fixed. Report and explain the effect on convergence speed (in epochs and in wall-clock time), gradient noise, and the generalization gap between training and validation performance.
- Implement a learning rate scheduler (e.g., `StepLR`, `CosineAnnealingLR`, or `ReduceLROnPlateau`) and compare its loss curve against a fixed learning rate baseline, identifying where the schedule helps or hurts convergence.
- Implement gradient accumulation to simulate a larger effective batch size than fits in memory at once. Verify that the accumulated gradient over several small batches matches, within tolerance, a single update computed directly on the equivalent large batch.
- Implement early stopping based on validation loss and gradient clipping to control exploding gradients. Demonstrate each mechanism on a case constructed specifically to trigger it (e.g., an unstable learning rate for clipping, or an overfitting run for early stopping).

Part D – Analysis & Documentation (15 Marks)

- Summarize every experiment run in Parts B and C — optimizer, batch size, schedule, and resulting metrics — in a single comparison table.
- State which optimizer/batch-size/scheduler combination gave the best trade-off between convergence speed and final generalization performance, citing the specific plots and numbers from your own runs as evidence.
- Reflect on at least two counter-intuitive findings from your experiments (e.g., a larger batch size not improving wall-clock convergence, or Adam not outperforming tuned SGD+momentum) and explain the likely mechanism behind each.
- Describe one concrete implementation 'gotcha' encountered while building these training loops manually (e.g., a missing `optimizer.zero_grad()`, a mis-scaled gradient-accumulation loss, or a scheduler stepped at the wrong frequency), how you diagnosed it, and how you fixed it.

Assessment Breakdown

Assessment Area	Marks
Loss Functions in Depth	20
Optimizers: SGD and Adam Internals	30
Epochs, Batches, and Training Loop Engineering	35
Analysis & Documentation	15
Total	100

Note: Building on the framework fluency established in Task 18, this task shifts the emphasis from using PyTorch's training utilities to understanding what they compute under the hood. Manual (from-scratch) implementations of SGD and Adam are mandatory in Part B and must be shown alongside their `torch.optim` equivalents — a working script that only calls `torch.optim` without the manual comparison will not satisfy this requirement. Document the PyTorch version used, all hyperparameters, and the random seed(s) fixed for each experiment, so that results are reproducible.